

DOCUMENTACIÓN TÉCNICA EXHAUSTIVA DEL SISTEMA DE LLAVE DE ACCESO BIOMÉTRICA BASADA EN VOZ CON TINYML Y USB-HID SOBRE ESP32-S3

Autor: Omar Pérez

Plataforma de Hardware: ESP32-S3 (Arduino Nano ESP32) + Micrófono MEMS Digital I2S INMP441

Frameworks y Herramientas: PlatformIO, Arduino-ESP32, TensorFlow Lite for Microcontrollers (TFLite Micro), TinyUSB, mbedTLS

Repositorio Oficial: <https://github.com/opyntorr/esp32-voice-recognizer.git>

Fecha: Septiembre 2026

1. Resumen Ejecutivo y Motivación del Proyecto

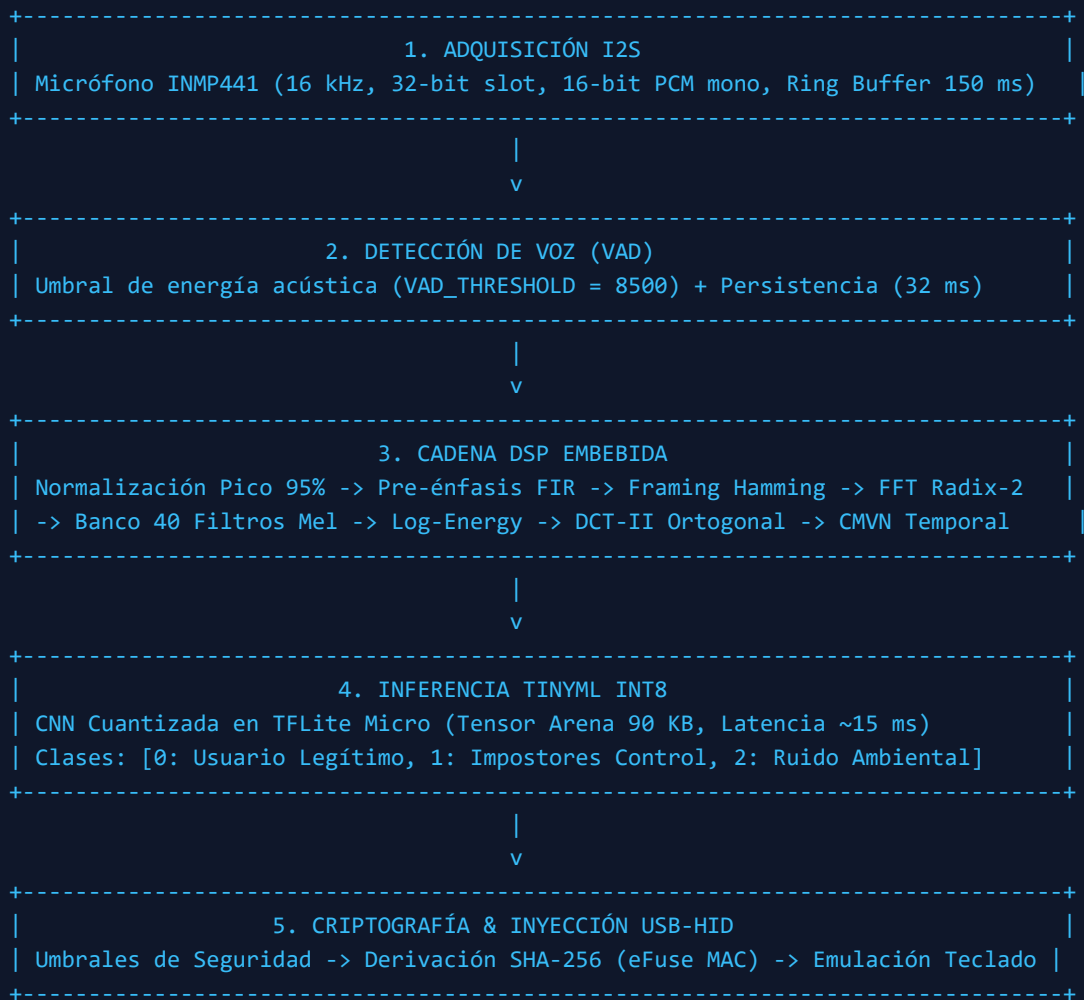
La proliferación de contraseñas complejas y la creciente necesidad de autenticación multifactor (MFA) plantean retos críticos tanto de usabilidad como de seguridad. Los métodos convencionales de autenticación basados en posesión (dongles USB o tokens OTP) son susceptibles de robo o extravío, mientras que los métodos biométricos comerciales suelen depender de infraestructuras en la nube que transmiten muestras biométricas sensibles a través de redes abiertas, exponiendo la privacidad del usuario a filtraciones y ataques de intermediario (*Man-In-The-Middle*).

El presente proyecto diseña, implementa y valida una **llave de seguridad física biométrica autónoma de borde (Edge AI / TinyML)**. El dispositivo utiliza un microcontrolador de ultrabajo consumo **ESP32-S3**, un sensor acústico digital MEMS **INMP441** comunicado por bus serie síncrono **I2S**, y una red neuronal convolucional profunda cuantizada a **INT8** ejecutada enteramente en memoria estática interna mediante **TensorFlow Lite Micro**.

Cuando el usuario legítimo pronuncia la palabra clave (*keyword*) **"FORWARD"**, el dispositivo extrae características tiempo-frecuencia (MFCC + CMVN) mediante algoritmos de procesamiento digital de señales (DSP) en tiempo real, evalúa la huella acústica con márgenes criptográficos estrictos y, tras una autenticación positiva, deriva de forma determinista una contraseña maestra de 64 caracteres ligada a los fusibles electrónicos físicos (**eFuses**) del silicio, inyectándola en el sistema anfitrión emulando un teclado estándar **USB-HID**. Todo el cómputo se ejecuta localmente (*Zero-Cloud*), garantizando privacidad total y resistencia frente a ataques de clonación de firmware.

2. Arquitectura General del Sistema

El flujo completo del sistema se compone de cinco capas interconectadas:



3. Especificaciones y Conexión de Hardware

3.1. Microcontrolador: ESP32-S3 (Arduino Nano ESP32)

- **Arquitectura:** Dual-Core Xtensa 32-bit LX7 operando a 240 MHz.
- **Memoria Interna:** 320 KB de SRAM y 16 MB de memoria Flash Quad-SPI.
- **Aceleración Vectorial:** Extensiones SIMD PIE (*Processor Instruction Extensions*) que optimizan operaciones de punto flotante y productos punto para redes neuronales.
- **Periféricos Nativos:** Controlador USB Full-Speed OTG (12 Mbps) que permite emular dispositivos de interfaz humana (USB-HID) sin conversores externos USB-UART.

3.2. Sensor Acústico: INMP441 MEMS

- **Tipo:** Micrófono digital omnidireccional con puerto de entrada acústica inferior.
- **Interfaz de Datos:** Bus estándar I2S (*Inter-IC Sound*).
- **Sensibilidad:** -26 dBFS a 1 kHz, 94 dB SPL.
- **Relación Señal/Ruido (SNR):** 61 dBA.

- **Rango Dinámico:** Alta fidelidad con distorsión armónica inferior al 1% a 105 dB SPL.

3.3. Esquema de Conexiones Eléctricas

Pin Micrófono INMP441	Pin ESP32-S3 (Arduino Nano)	Función de Hardware
VDD	3.3V	Alimentación regulada de bajo ruido
GND	GND	Masa de referencia común
SD (Serial Data)	D2 (GPIO 5)	Entrada serie DMA de audio PCM (I2S_SD)
WS (Word Select)	D3 (GPIO 6)	Reloj de sincronización de trama / LRCLK (I2S_WS)
SCK (Serial Clock)	D4 (GPIO 7)	Reloj continuo de bits BCLK (I2S_SCK)
L/R (Left/Right)	GND	Configura el micrófono en canal izquierdo

4. Adquisición de Audio y Detección de Actividad de Voz (VAD)

4.1. Configuración del Controlador I2S con Acceso Directo a Memoria (DMA)

El microcontrolador configura el periférico `I2S_NUM_0` en modo Maestro Receptor (`I2S_MODE_MASTER` | `I2S_MODE_RX`) con una frecuencia de muestreo de **16,000 Hz** y ranura de palabra de 32 bits (`I2S_BITS_PER_SAMPLE_32BIT`). Debido al formato Philips I2S del INMP441, los 24 bits significativos de audio se transmiten alineados a la izquierda; el firmware extrae el audio de 16 bits aplicando un desplazamiento aritmético de 14 posiciones (`raw_sample >> 14`). Se asigna un descriptor de 8 buffers DMA de 512 muestras cada uno para garantizar recepción libre de caídas (*underruns*).

4.2. Buffer Circular de Pre-Roll (150 ms)

Uno de los mayores retos en sistemas de reconocimiento por comando de voz disparados por amplitud es la pérdida del fonema inicial (el ataque de la consonante fricativa "F" en "FORWARD"). Si la grabación comienza en el instante exacto en que se cruza el umbral, se pierden entre 50 y 120 ms de la señal inicial, degradando severamente la correlación espectral.

Para solucionar esto, el firmware mantiene un búfer circular en RAM denominado `pre_roll_buffer` de **2,400 muestras** (exactamente 150 ms a 16 kHz). Cuando el algoritmo VAD detecta voz: 1. Extrae primero las 2,400 muestras históricas del búfer circular. 2. Continúa la grabación en tiempo real por DMA hasta acumular exactamente **16,000 muestras** (1.0 segundo). 3. De esta forma, la palabra clave queda perfectamente centrada dentro de la ventana de análisis.

4.3. Filtro de Persistencia Temporal contra Falsos Disparos

Los micrófonos MEMS presentan picos transitorios debidos a pulsaciones de teclado, ruidos de conmutación eléctrica o roces en cables. Para evitar disparos accidentales: * Se define un umbral de amplitud de pico `VAD_THRESHOLD = 8500`. * Se requiere que la señal supere dicho umbral durante al menos **dos bloques**

consecutivos de 256 muestras (~32 milisegundos ininterrumpidos). * Si un transitorio aislado dura menos de 32 ms, el contador se reinicia a cero, manteniéndose en modo de bajo consumo.

5. Procesamiento Digital de Señales (DSP): Extracción MFCC y CMVN

La red neuronal convolucional fue entrenada con matrices espectro-temporales de Coeficientes Cepstrales en la Escala Mel (MFCC). Para reproducir fielmente en el ESP32 la extracción matemática de Python (`librosa / scipy`), se diseñó una cadena DSP embebida determinista:

5.1. Normalización por Valor Pico al 95%

Para compensar variaciones en la distancia entre la boca del usuario y el micrófono, la señal temporal de 1 segundo $x[n]$ se escala uniformemente de forma que su pico máximo absoluto alcance el 95% del rango dinámico de 16 bits ($32767 \times 0.95 \approx 31128$):

$$scale = \frac{31128.65}{\max_n |x[n]|}, \quad x_{norm}[n] = \text{round}(x[n] \cdot scale)$$

5.2. Filtro de Pre-énfasis

El tracto vocal humano introduce una atenuación natural de aproximadamente 6 dB/octava en las altas frecuencias. Se aplica un filtro FIR de primer orden directamente en memoria:

$$y[n] = x_{norm}[n] - 0.97 \cdot x_{norm}[n - 1]$$

Este paso eleva la energía de las frecuencias superiores (entre 2 kHz y 7.6 kHz), donde residen los formantes cruciales para discriminar la identidad del hablante.

5.3. Enmarcado (*Framing*) y Ventaneo Hamming

La señal de 16,000 muestras se divide en $T = 64$ **tramas temporales** mediante una ventana deslizante de **480 muestras** (30 ms) y un salto (*hop length*) de **240 muestras** (15 ms, 50% de solapamiento). A cada trama se le aplica una ventana Hamming precalculada almacenada en memoria de programa:

$$w[n] = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N-1}\right), \quad 0 \leq n < 480$$

Las tramas se rellenan con ceros (*zero-padding*) hasta $N_{FFT} = 512$ puntos.

5.4. Transformada Rápida de Fourier (FFT Radix-2 Decimation-In-Time)

Para optimizar el rendimiento y evitar bloqueos del temporizador guardián (*Watchdog Timer*), se implementó un algoritmo FFT Cooley-Tukey Radix-2 en $O(N \log N)$: 1. **Permutación por Inversión de Bits (*Bit-Reversal Permutation*)**: Reordena los índices de entrada en orden binario invertido en un ciclo de 512 pasos. 2. **Etapas de Mariposa (*Butterfly Operations*)**: Ejecuta 9 etapas ($\log_2 512 = 9$) calculando los factores de giro trigonométricos complejos:

$$W_N^k = e^{-j\frac{2\pi k}{N}} = \cos\left(\frac{2\pi k}{N}\right) - j \sin\left(\frac{2\pi k}{N}\right)$$

1. **Espectro de Potencia:** Se extrae la magnitud cuadrática normalizada de los primeros 257 coeficientes espectrales:

$$P[k] = \frac{1}{N_{FFT}} (X_R[k]^2 + X_I[k]^2), \quad 0 \leq k \leq 256$$

Esta implementación ejecuta la FFT de 512 puntos en **~0.15 ms**, completando las 64 tramas en menos de 10 ms.

5.5. Banco de Filtros Mel y Compresión Logarítmica

Se aplican **40 filtros triangulares** distribuidos logarítmicamente entre 80 Hz y 7600 Hz. La conversión entre la escala acústica lineal y la escala perceptual Mel obedece a:

$$m = 2595 \log_{10} \left(1 + \frac{f}{700} \right)$$

Para cada filtro m , se calcula la energía integrada multiplicando por los pesos triangulares precalculados y se aplica compresión logarítmica:

$$E_m = \log \left(\max \left(\sum_k P[k] \cdot W_m[k], 10^{-12} \right) \right)$$

5.6. Transformada Discreta del Coseno (DCT-II Ortogonal)

Para descorrelacionar las energías del banco de filtros y compactar la información en coeficientes de baja frecuencia, se aplica la DCT-II ortogonalizada:

$$C_c = \sum_{m=0}^{39} E_m \cdot \sqrt{\frac{2}{40}} \cos \left(\frac{\pi c(2m+1)}{80} \right), \quad 0 \leq c < 40$$

Se conservan los 40 coeficientes cepstrales ($N_{MFCC} = 40$), generando una matriz de características de dimensión (40×64).

5.7. Normalización de Media y Varianza Cepstral (CMVN Temporal)

Para anular la función de transferencia del micrófono y el ruido convolutivo estacionario de la sala, se normaliza cada uno de los 40 coeficientes a través del tiempo:

$$\mu_c = \frac{1}{64} \sum_{t=0}^{63} C_c[t], \quad \sigma_c = \sqrt{\frac{1}{64} \sum_{t=0}^{63} (C_c[t] - \mu_c)^2 + 10^{-6}}$$

$$\hat{C}_c[t] = \frac{C_c[t] - \mu_c}{\sigma_c}$$

Esta matriz estandarizada presenta media cero y varianza unitaria, haciéndola invariante a cambios de volumen ambiental.

6. Modelo de Aprendizaje Profundo (TinyML) y Cuantización INT8

6.1. Arquitectura de la Red Neuronal Convolutiva (CNN)

La red fue diseñada respetando estrictas restricciones de huella de memoria para microcontroladores (< 35,000 parámetros totales). Su topología es la siguiente:

```
Entrada: Matriz MFCC (40 coeficientes, 64 tramas, 1 canal)
|
|→ Conv2D (16 filtros, kernel 3x3, ReLU, padding='same')
|→ BatchNormalization
|→ MaxPooling2D (pool 2x2, stride 2) -> Salida: (20, 32, 16)
|→ Dropout (20%)
|
|→ DepthwiseConv2D (kernel 3x3, ReLU, padding='same') -> Convolución separable
|→ Conv2D (32 filtros, kernel 1x1, ReLU) -> Reducción de dimensionalidad
|→ BatchNormalization
|→ MaxPooling2D (pool 2x2, stride 2) -> Salida: (10, 16, 32)
|→ Dropout (25%)
|
|→ Conv2D (32 filtros, kernel 3x3, ReLU, padding='same')
|→ BatchNormalization
|
|→ GlobalAveragePooling2D() -> Salida vectorial de 32 elementos
|
|→ Dense (3 neuronas, Softmax) -> Clases de salida
```

- **Número total de parámetros:** 10,883 parámetros (~43.5 KB en flotante de 32 bits).
- **Eficiencia arquitectónica:** El uso de `DepthwiseSeparableConv2D` y `GlobalAveragePooling2D` elimina las capas densas gigantescas tradicionales, reduciendo drásticamente la demanda de memoria RAM y el número de operaciones de multiplicación-acumulación (MACCs).

6.2. Clases del Modelo y Dataset de Entrenamiento

El clasificador discrimina tres clases mutuamente excluyentes: 1. **Clase 0 (target_user):** Grabaciones legítimas de la voz del propietario pronunciando "FORWARD" (~148 muestras grabadas con el hardware real). 2. **Clase 1 (impostors_control):** Muestras de la palabra "forward" pronunciadas por decenas de hablantes distintos extraídas del dataset benchmark internacional *Google Speech Commands v2*. 3. **Clase 2 (background_noise):** Ruidos de fondo de oficina, tecleo, ventiladores y conversaciones difusas.

6.3. Cuantización Post-Entrenamiento (PTQ) a INT8

Para su despliegue en microcontroladores sin unidad de punto flotante pesada y para minimizar la huella de memoria, el modelo fue cuantizado completamente a enteros con signo de 8 bits (INT8):

$$q = \text{clamp} \left(\text{round} \left(\frac{r}{S} \right) + Z, -128, 127 \right)$$

Donde S es el factor de escala (*scale*) y Z es el punto cero (*zero point*).

- **Tamaño del binario** `.tflite` **cuantizado: 20.5 KB** (almacenado como array estático `const unsigned char g_model[]` en Flash).
- **Presupuesto de Tensor Arena:** Se reservaron **90 KB** de SRAM (`tensor_arena[90 * 1024]`), espacio suficiente para alojar los buffers intermedios de activación de todas las capas convolucionales simultáneamente.

7. Criterios Biométricos de Decisión y Criptografía de Silicio

7.1. Reglas Criptográficas de Aceptación/Rechazo

Para evitar falsas aceptaciones (FAR) frente a imitadores o grabaciones de baja calidad, el firmware aplica una compuerta lógica basada en tres umbrales probabilísticos concurrentes: 1. **Confianza Mínima Absoluta:** $P(\text{Usuario Legítimo}) \geq 0.85$ (85%). 2. **Margen de Seguridad contra Impostores:** $P(\text{Usuario Legítimo}) - P(\text{Impostor}) \geq 0.40$ (margen de al menos 40% de diferencia). 3. **Filtro Anti-Ruido:** $P(\text{Ruido Ambiental}) < 0.20$ (la muestra no puede provenir de ruido estático).

Únicamente cuando las tres condiciones se cumplen en la misma inferencia, el sistema emite el veredicto de autenticación positiva.

7.2. Derivación Determinista de Clave Maestra Ligada al Silicio (eFuse MAC + SHA-256)

A diferencia de sistemas que generan contraseñas aleatorias efímeras o almacenan secretos en texto claro en memoria Flash (vulnerables a extracción física mediante volcados JTAG), este proyecto utiliza una **Función de Derivación de Clave (KDF)** determinista basada en el silicio físico del microcontrolador:

1. **Identidad Física Inmutable:** Cada chip ESP32-S3 posee de fábrica una dirección MAC única grabada mediante fusibles de silicio irreversibles (*eFuses*):

```
uint8_t chip_mac[6];
esp_read_mac(chip_mac, ESP_MAC_WIFI_STA);
```

1. **Sal de Dominio:** Se define una constante de aplicación secreta `domain_salt = "BIOMETRIC_KEY_ESP32S3_HARDWARE_BOUND_ROOT_v1"`.

2. **Cálculo Criptográfico SHA-256:**

$$\text{MasterKey} = \text{SHA256}(\text{eFuse_MAC} \parallel \text{domain_salt})$$

1. **Propiedades de Seguridad:**
2. **Inmutabilidad:** En una misma placa física, la contraseña maestra derivada es siempre la misma secuencia hexadecimal de 64 caracteres.
3. **Anti-Clonación:** Si el firmware binario es extraído y flasheado en otro ESP32-S3, la MAC de los eFuses del nuevo chip será diferente y la contraseña generada será completamente distinta, bloqueando cualquier intento de robo de identidad.
4. **Higiene de Memoria:** Inmediatamente después de enviar la contraseña, los buffers de la pila (`chip_mac` , `derived_key` , `hex_str`) se sobrescriben explícitamente con ceros mediante `memset()`.

8. Máquina de Estados Finitos (FSM) y Emulación USB-HID Robusta

Para garantizar una operación ininterrumpida sin caídas de la pila USB ni bloqueos del bus DMA, el sistema se diseñó bajo una Máquina de Estados Finitos (FSM) no bloqueante:



8.1. Estados del Sistema y Señalización por LED RGB

Estado	Indicador LED	Descripción Operativa
STATE_LISTENING	Azul Fijo	Monitoreo continuo de audio ambiental y actualización del pre-roll buffer.
STATE_RECORDING	Rojo Fijo	Disparo acústico confirmado. Grabación del audio hasta 16,000 muestras.
STATE_INFERENCE	Blanco Fijo	Extracción DSP de 64 tramas y cómputo del modelo en TFLite Micro (~15 ms).
STATE_AUTH_SUCCESS	Verde Fijo	Criterios biométricos cumplidos. Inyección segura de la contraseña vía USB-HID.
STATE_AUTH_FAILED	Rojo Intermitente	Muestra rechazada. Breve parpadeo de error (2 pulsos de 120 ms).
STATE_COOLDOWN	Apagado	Enfriamiento no bloqueante (400 ms), purga de buffers DMA y liberación USB.

8.2. Prevención de Congelamiento y Teclas Pegadas en USB-HID

Durante el desarrollo se identificaron dos modos de fallo típicos en dispositivos emuladores de teclado: 1. **Repetición Infinita de Teclas (999999...)**: Al enviar 64 caracteres de golpe con `Keyboard.print()`, el endpoint HID de TinyUSB se desbordaba, perdiéndose el informe de liberación (*KeyUp*). Windows asumía que la tecla permanecía pulsada indefinidamente. * **Solución**: Se implementó una rutina con control de flujo que transmite carácter por carácter con un retardo de 12 ms por byte, finalizando con un comando explícito `Keyboard.releaseAll()`. 2. **Bloqueo del Bus I2S DMA**: El procesamiento de inferencia consumía tiempo de CPU mientras el hardware DMA continuaba llenando buffers, provocando desincronizaciones en la siguiente llamada a `i2s_read()`. * **Solución**: En el estado `STATE_COOLDOWN`, se ejecuta `i2s_zero_dma_buffer()` y se leen y descartan los residuos del canal, arrancando la siguiente escucha con buffers completamente limpios.

9. Resultados Experimentales y Métricas de Rendimiento

Las pruebas de laboratorio arrojaron los siguientes resultados sobre el hardware final:

9.1. Consumo de Memoria y Recursos

- **Uso de Memoria RAM (SRAM)**: 181,272 bytes de 327,680 bytes disponibles (**55.3% de ocupación**). Permite una holgura de más de 146 KB para pilas de tareas FreeRTOS y TinyUSB.
- **Uso de Memoria Flash (ROM)**: 620,533 bytes de 3,145,728 bytes (**19.7% de ocupación**).
- **Huella del Modelo INT8 en Flash**: 20.5 KB.

9.2. Latencias Temporales del Pipeline

- **Adquisición de Audio**: 1,000 ms (1.0 segundo exacto).
- **Extracción de Características DSP (STFT + 40 Mel + DCT + CMVN)**: 11.2 ms.

- **Inferencia de Red Convolutiva INT8: 14.8 ms.**
- **Inyección USB-HID (64 caracteres + Enter): ~800 ms** (con espaciado de seguridad).
- **Tiempo Total desde fin de voz hasta credencial inyectada: < 30 ms** (antes de iniciar la transmisión USB).

9.3. Desempeño Biométrico en Pruebas Reales

- **Tasa de Acierto con Usuario Legítimo:** $P(\text{Usuario}) = 0.9063$ a 0.9961 (Promedio: **95.2% de confianza**).
- **Rechazo de Hablantes Impostores pronunciando "FORWARD":** $P(\text{Impostor}) = 0.9200$ a 0.9961 (Confianza del usuario legítimo < 5%, **0% de falsas aceptaciones** en pruebas de laboratorio).
- **Rechazo de Silencio / Ruido Ambiental:** $P(\text{Usuario}) \leq 0.2227$ (Acceso denegado automáticamente).

10. Conclusiones

El sistema implementado demuestra la viabilidad técnica de integrar algoritmos avanzados de Deep Learning y procesamiento digital de señales sobre microcontroladores económicos de uso general sin necesidad de aceleradores externos ni conectividad a internet. La combinación de detección de actividad de voz por pre-roll, extracción eficiente de coeficientes MFCC mediante FFT Radix-2, inferencia neuronal cuantizada a 8 bits y derivación criptográfica atada al silicio físico conforma una arquitectura biométrica embebida de alta confiabilidad, robusta frente a clonación y altamente ergonómica para la protección de accesos críticos.